Name: Anirudh Vineet

Batch: T23
Roll no: 2401124
Subject: ASD & Devops

Experiment No: 3

Aim: To deploy Multi-service application using Docker Compose

2. Theoretical Background

What is Docker Compose?

Docker Compose is an orchestration tool provided by Docker that allows developers to define and manage multi-container applications using a single YAML configuration file (docker-compose.yml). Instead of starting, networking, and linking individual containers manually using complex command-line arguments, Docker Compose allows you to configure all application services, network bridges, volume mounts, and environment variables in one structured file.

Key Concepts

Service: An application component running in its own containerized environment (e.g., Node.js API, Nginx web server, PostgreSQL database).

Network: An isolated internal bridge network that allows containers within the same stack to discover and communicate with each other using service names as domain names.

Volume: A persistent storage mechanism managed by Docker that allows database files or cache data to persist even when containers are stopped, restarted, or deleted.

Environment Variables (.env): A configuration file used to pass runtime parameters, credentials, and port configurations dynamically into containers without hardcoding them into source code.

3. Summary of Commands Executed

Below is the structured list of commands implemented throughout the lab, organized by their usage and lifecycle stage.

Category	Command Implemented	Description / Purpose
File Creation	notepad docker-compose.yml	Opens Windows Notepad to create and edit the YAML configuration file in the current working directory.
File Creation	notepad .env	Creates the environment variable file to store dynamic settings like database passwords and port numbers.
Directory Setup	mkdir backend mkdir frontend	Creates structured folders to hold backend API source code, frontend assets, and custom Dockerfiles.
Troubleshooting	ren backend\Dockerfile.txt Dockerfile	Fixes Windows Notepad automatically appending .txt extensions to extensionless Dockerfiles.
Deployment	docker compose up -d --build	Forces Docker to rebuild custom container images, pull required registry images, and launch all services in detached mode.
Status Check	docker compose ps	Displays the current status, container names, health, and mapped host ports of all active services.
Testing	curl http://localhost:3000	Sends an HTTP request to test connectivity and output from the application endpoint in Command Prompt.
Log Inspection	docker compose logs -f	Streams aggregated live console logs from all containers for real-time output and error debugging.
Database Access	docker compose exec postgres psql -U admin -d app_db	Opens an interactive PostgreSQL database terminal inside the running postgres container.
Cache Access	docker compose exec redis redis-cli	Launches the Redis CLI inside the running cache container to inspect stored keys and visit counters.

Cleanup	docker compose down -v	Stops all running services, destroys container instances, deletes created bridges, and wipes associated data volumes.

4. Step-by-Step Implementation Workflow

Step 1: Setting up Project Structure and Environment Variables

First, we configured the project workspace and stored configuration parameters inside a .env file to keep configurations separated from core application code.

```
FRONTEND_PORT=3000
BACKEND_PORT=5000
DB_PORT=5432
REDIS_PORT=6379
POSTGRES_USER=admin
POSTGRES_PASSWORD=secretpassword
POSTGRES_DB=app_db
```

Step 2: Configuring the Multi-Service YAML Stack

Next, we created docker-compose.yml to define our 3-tier setup combining frontend, backend, caching, and database services.

```
services:
  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    container_name: app_frontend
    ports:
      - "${FRONTEND_PORT}:80"
    depends_on:
      - backend
    networks:
      - app-network

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: app_backend
    ports:
      - "${BACKEND_PORT}:5000"
    environment:
      - POSTGRES_USER=${POSTGRES_USER}
      - POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
      - POSTGRES_DB=${POSTGRES_DB}
    depends_on:
      - postgres
      - redis
    networks:
      - app-network

  postgres:
    image: postgres:15-alpine
    container_name: app_postgres
    ports:
      - "${DB_PORT}:5432"
    environment:
      - POSTGRES_USER=${POSTGRES_USER}
```

```
      - POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
      - POSTGRES_DB=${POSTGRES_DB}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - app-network

  redis:
    image: redis:alpine
    container_name: app_redis
    ports:
      - "${REDIS_PORT}:6379"
    networks:
      - app-network

networks:
  app-network:
    driver: bridge

volumes:
  postgres_data:
```

Step 3: Building and Deploying the Stack

We launched the stack using the build flag to ensure all local Dockerfiles were compiled into fresh container images.

```
C:\Users\15L\docker-compose-lab> docker compose up -d --build
```

Output:

```
C:\Users\15L\docker-compose-lab>mkdir backend

C:\Users\15L\docker-compose-lab>notepad backend\package.json

C:\Users\15L\docker-compose-lab>notepad backend\server.js

C:\Users\15L\docker-compose-lab>notepad backend\Dockerfile

C:\Users\15L\docker-compose-lab>mkdir frontend

C:\Users\15L\docker-compose-lab>notepad frontend\index.html

C:\Users\15L\docker-compose-lab>notepad frontend\Dockerfile

C:\Users\15L\docker-compose-lab>notepad docker-compose.yml

✓ Image   docker-compose-lab-frontend         Built                14.9s
✓ Image   docker-compose-lab-backend          Built                14.9s
✓ Volume  docker-compose-lab_postgres_data    Created               0.0s
✓ Container  app_postgres                     Created               0.1s
✓ Container  redis_cache                      Recreated             0.4s
✓ Container  app_backend                      Created               0.1s
✓ Container  app_frontend                     Created               0.1s

C:\Users\15L\docker-compose-lab>docker compose ps
NAME           IMAGE                       COMMAND                  SERVICE    CREATED
STATUS
app_backend    docker-compose-lab-backend  "docker-entrypoint..."   backend    12 seconds ago
Up 11 seconds  0.0.0.0:5000->5000/tcp
app_frontend   docker-compose-lab-frontend "/docker-entrypoint..." frontend   12 seconds ago
Up 11 seconds  0.0.0.0:3000->80/tcp
```

```
app_postgres    postgres:15-alpine          "docker-entrypoint.s..." postgres    13 seconds ago
Up 11 seconds   0.0.0.0:5432->5432/tcp
app_redis       redis:alpine                "docker-entrypoint.s..." redis       13 seconds ago
Up 11 seconds   0.0.0.0:6379->6379/tcp
```

Log Inspection and Verification

```
C:\Users\15L\docker-compose-lab>docker compose logs -f
app_redis      | Starting Redis Server
app_frontend   | /docker-entrypoint.sh: Configuration complete; ready for start up
app_postgres   | The files belonging to this database system will be owned by user "postgres".
app_backend    | > backend@1.0.0 start
app_backend    | > node server.js
app_backend    | Backend listening on port 5000
app_redis      | Redis version 8.8.1, bits=64, just started
```

Database and Cache Access

```
C:\Users\15L\docker-compose-lab>docker compose exec postgres psql -U admin -d app_db
psql (15.18)
Type "help" for help.
app_db=# \q

C:\Users\15L\docker-compose-lab>docker compose exec redis redis-cli
127.0.0.1:6379> KEYS *
1) "visits"
127.0.0.1:6379> exit
```

Stopping and Restarting Services

```
C:\Users\15L\docker-compose-lab>docker compose stop
✓ Container  app_frontend     Stopped    0.3s
✓ Container  app_backend      Stopped    0.8s
✓ Container  app_redis        Stopped    0.4s
✓ Container  app_postgres     Stopped    0.3s

C:\Users\15L\docker-compose-lab>docker compose start
[+] start 4/4
```

Step 4: Cleanup

```
C:\Users\15L\docker-compose-lab>docker compose down
✓ Container  app_frontend     Removed    0.3s
✓ Container  app_backend      Removed    0.7s
✓ Container  app_postgres     Removed    0.3s
✓ Container  app_redis        Removed    0.4s

C:\Users\15L\docker-compose-lab>docker compose down -v
✓ Volume   docker-compose-lab_postgres_data     Removed    0.0s
```

5. Conclusion

In this lab, we successfully built and deployed a multi-service application architecture using Docker Compose. By consolidating service definitions, network bridges, and volumes into a single docker-compose.yml file, we eliminated the need to run long, manual Docker commands for each service. Additionally, we addressed real-world deployment challenges such as port binding conflicts, file extension issues in Windows CMD, and image pulling over Docker Hub. The lab highlights how Docker Compose simplifies development workflows, improves local environment parity, and enables rapid deployment of complex microservices setups with minimal configuration overhead.